

**Instituto Politécnico Nacional**
Escuela Superior de Cómputo




Ingeniería en Inteligencia Artificial

Academia de Sistemas Distribuidos

Cómputo Paralelo

6BM1

# Práctica 1: Plataformas y Herramientas

Equipo 9

Arce Padilla Carlos - 2024630998
Montes Olguin José Alberto - 2024631047
Salinas Padilla Rubén Iván - 2024630873

Plan 2020

Profesor: Rodrigo Ojeda Santillán

08 de octubre de 2026

## Objetivo

Configurar y verificar un entorno de trabajo para cómputo paralelo mediante herramientas de desarrollo en Linux, así como evaluar el rendimiento de diferentes implementaciones de multiplicación de matrices en C, Python y NumPy. Esto mediante la medición de tiempos de ejecución, el cálculo de GFLOPS y el análisis de las optimizaciones del compilador, con la finalidad de establecer una línea base secuencial que permita realizar futuras comparaciones de rendimiento.

## Introducción

El cómputo paralelo permite aprovechar los recursos de procesamiento disponibles para ejecutar operaciones de manera más eficiente. Sin embargo, antes de implementar técnicas de paralelización, es importante conocer las características del hardware, las herramientas de desarrollo y los métodos adecuados para evaluar el rendimiento de un programa.

En esta práctica se realizó la configuración de un entorno Linux mediante WSL2, utilizando herramientas como GCC, G++, Python, Git y OpenMP. Posteriormente, se identificaron las características de los equipos utilizados y se realizaron pruebas de multiplicación de matrices con diferentes implementaciones en C, Python puro y NumPy, considerando distintos tamaños de entrada y niveles de optimización.

Para evaluar el rendimiento, se registraron los tiempos de ejecución mediante múltiples repeticiones y se calcularon medidas estadísticas como el promedio y la desviación estándar, además del rendimiento expresado en GFLOPS. Finalmente, se analizaron las diferencias entre las implementaciones, el efecto de las optimizaciones del compilador y el aprovechamiento de los recursos de procesamiento, estableciendo una referencia para futuras prácticas de cómputo paralelo.

## Desarrollo

### A) Configuración

Cada integrante preparó su entorno en Ubuntu sobre WSL2 y verificó el compilador, Python, NumPy, Git y el soporte de OpenMP con los comandos de la Parte A. La Tabla 1 resume lo obtenido; la salida completa de la terminal de cada integrante está en el Anexo B.

*Tabla 1. Verificación del entorno de trabajo de cada integrante.*

| Integrante | gcc / g++ | Python / NumPy | Git | OpenMP | hello_omp |
|---|---|---|---|---|---|
| Salinas Padilla Rubén Iván | 15.2.0 | 3.14.4 / 2.5.3 | 2.53.0 | _OPENMP 201511 | 4 hilos (valor por defecto) |
| Arce Padilla Carlos | 15.2.0 | 3.14.4 / 2.5.3 | 2.53.0 | _OPENMP 201511 | 4 hilos (OMP_NUM_THREADS=4) |
| Montes Olguin José Alberto | 13.3.0 | 3.12.3 / 2.5.3 | 2.43.0 | _OPENMP 201511 | 1 y 4 hilos (OMP_NUM_THREADS=1 y 4) |

### B) Fichas técnicas

*Tabla 2. Fichas técnicas de las máquinas del equipo.*

| Campo | Salinas Padilla Rubén Iván | Arce Padilla Carlos | Montes Olguin José Alberto |
|---|---|---|---|
| **Modelo de CPU** | 11th Gen Intel(R) Core(TM) i3-1115G4 @ 3.00 GHz | 12th Gen Intel(R) Core(TM) i5-12400F | AMD Ryzen 9 5900HS with Radeon Graphics |
| **Núcleos físicos / hilos lógicos** | 2 / 4 | 6 / 12 | 8 / 16 |
| **Frecuencia base / turbo** | 3.00 GHz / 4.10 GHz (especificación de Intel) | 2.50 GHz / 4.40 GHz (especificación de Intel) | 3.0 GHz / 4.6 GHz (especificación de AMD) |
| **Caché L1d / L2 / L3** | 48 KiB por núcleo / 1.25 MiB por núcleo / 6 MiB compartida | 48 KiB por núcleo / 1.25 MiB por núcleo / 18 MiB compartida | 32 KiB por núcleo / 512 KiB por núcleo / 16 MiB compartida |

| Campo | Salinas Padilla Rubén Iván | Arce Padilla Carlos | Montes Olguin José Alberto |
|---|---|---|---|
| **Memoria RAM** | 3.7 GiB disponibles para WSL2 | 7.7 GiB disponibles para WSL2 | 11 GiB disponibles para WSL2 (24 GB instalados) |
| **Extensiones vectoriales** | SSE4.2, AVX, AVX2, AVX-512F, FMA | SSE4.2, AVX, AVX2, FMA | SSE4.2, AVX, AVX2, FMA |
| **Sistema operativo y kernel** | Ubuntu sobre WSL2 / kernel por verificar | Ubuntu sobre WSL2 / kernel por verificar | Ubuntu 24.04.5 LTS sobre WSL2 / kernel 6.18.40.1-microsoft-standard-WSL2 |
| **Compilador** | GCC 15.2.0 / G++ 15.2.0 | GCC 15.2.0 / G++ 15.2.0 | GCC 13.3.0 / G++ 13.3.0 |
| **Python / NumPy / BLAS** | Python 3.14.4 / NumPy 2.5.3 / OpenBLAS 0.3.34.106.0 | Python 3.14.4 / NumPy 2.5.3 / OpenBLAS 0.3.34.106.0 | Python 3.12.3 / NumPy 2.5.3 / OpenBLAS 0.3.34.106.0 |
| **GPU (si hay)** | No se detectó GPU NVIDIA disponible mediante nvidia-smi en WSL | Ninguna tarjeta gráfica NVIDIA local en el entorno de WSL | NVIDIA GeForce RTX 3060 Laptop GPU (6 GB), detectada con nvidia-smi en WSL2 |
| **Condiciones** | Ejecución dentro de Ubuntu sobre WSL2, laptop conectada a la corriente; antes de la corrida reportada se cerraron las aplicaciones innecesarias. | Pruebas ejecutadas en entorno local sobre WSL2, con el equipo conectado a la corriente y condiciones estándar de escritorio | Ejecución en Ubuntu sobre WSL2, laptop conectada a la corriente, plan de energía "Turbo" |

Para las siguientes secciones se usó el equipo de Salinas Padilla Rubén Iván.

### C) Metodología de medición

Para evaluar el rendimiento de las diferentes implementaciones de multiplicación de matrices se siguió el protocolo de medición establecido en la práctica. Como métrica principal se utilizó el tiempo de pared (wall-clock), debido a que representa el tiempo real transcurrido durante la ejecución y es el utilizado posteriormente para calcular aceleraciones o *speedup*.

En la implementación en C, el tiempo se obtuvo mediante “clock_gettime(CLOCK_MONOTONIC, &ts);” mientras que en implementaciones de Python “time.perf_counter()”. Estos relojes son monotónicos y de alta resolución, por lo que resultan adecuados para medir tiempos de ejecución. La guía establece específicamente estos mecanismos para las mediciones en C y Python.

Antes de comenzar las mediciones se realizó una ejecución de calentamiento, cuyo resultado fue descartado. Esto permite reducir efectos relacionados con cachés frías, carga inicial de bibliotecas, asignación de memoria y cambios iniciales en la frecuencia del procesador. Posteriormente se realizaron cinco repeticiones por cada configuración, calculando el promedio y la desviación estándar de los tiempos obtenidos. Este procedimiento corresponde al protocolo requerido por la práctica.

Para la implementación en C se evaluaron tres niveles de optimización, que el archivo Makefile compila mediante gcc -O0 -std=c11, gcc -O2 -std=c11 y gcc -O3 -march=native -std=c11 (ejecutables matmul_O0, matmul_O2 y matmul_O3). Los tamaños evaluados fueron n = 256, 512 y 1024 en C; n = 64, 128 y 256 en Python puro, y n = 256, 512, 1024 y 2048 en NumPy.

Para NumPy se realizaron dos configuraciones: la ejecución normal, en la que BLAS puede utilizar varios hilos, y una ejecución restringida a un solo hilo mediante OMP_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1 MKL_NUM_THREADS=1.

La ejecución completa de las pruebas se realizó desde el directorio src mediante “bash ../scripts/run_all.sh”. Posteriormente se generó la tabla resumen y las gráficas desde el directorio practica1 mediante “python3 scripts/plot.py”.

El script run_all.sh automatiza la compilación y las cinco repeticiones de cada configuración, mientras que plot.py agrupa los resultados y calcula el promedio, la desviación estándar y el rendimiento medio en GFLOPS. Las mediciones se realizaron procurando mantener condiciones estables: equipo conectado a la corriente y

reducción de aplicaciones y procesos en segundo plano que pudieran competir por CPU y memoria. En una primera ejecución se observó una variabilidad elevada, por lo que las mediciones fueron repetidas después de cerrar aplicaciones innecesarias.

### D) Resultados

La Tabla 3 presenta los resultados obtenidos en la segunda ejecución de las pruebas. Para cada configuración se muestra el tiempo promedio junto con su desviación estándar y el rendimiento promedio expresado en GFLOPS.

*Tabla 3. Tiempo promedio ± desviación estándar y rendimiento en GFLOPS de cada variante (equipo de Salinas Padilla Rubén Iván).*

| Variante | n | Tiempo prom. ± desv. est. [s] | GFLOPS |
|---|---|---|---|
| c_O0 | 256 | 0.043268 ± 0.002535 | 0.777 |
| c_O0 | 512 | 0.430791 ± 0.014688 | 0.624 |
| c_O0 | 1024 | 5.313345 ± 0.066424 | 0.404 |
| c_O2 | 256 | 0.015727 ± 0.001190 | 2.143 |
| c_O2 | 512 | 0.244511 ± 0.006881 | 1.099 |
| c_O2 | 1024 | 3.231043 ± 0.024529 | 0.665 |
| c_O3 | 256 | 0.015313 ± 0.001109 | 2.200 |
| c_O3 | 512 | 0.270291 ± 0.012204 | 0.995 |
| c_O3 | 1024 | 3.078917 ± 0.025515 | 0.698 |
| NumPy | 256 | 0.000450 ± 0.000107 | 77.226 |
| NumPy | 512 | 0.003281 ± 0.000460 | 83.049 |
| NumPy | 1024 | 0.032216 ± 0.002797 | 67.078 |
| NumPy | 2048 | 0.239884 ± 0.054034 | 74.167 |
| NumPy 1 hilo | 256 | 0.000742 ± 0.000164 | 46.851 |
| NumPy 1 hilo | 512 | 0.005882 ± 0.001120 | 46.790 |
| NumPy 1 hilo | 1024 | 0.042481 ± 0.001108 | 50.580 |
| NumPy 1 hilo | 2048 | 0.313831 ± 0.001893 | 54.744 |
| Python puro | 64 | 0.007703 ± 0.000160 | 0.0681 |
| Python puro | 128 | 0.064142 ± 0.002234 | 0.0655 |
| Python puro | 256 | 0.496554 ± 0.009684 | 0.0676 |

En la gráfica de tiempo (Figura 1, izquierda) puede observarse que NumPy presenta los menores tiempos de ejecución, seguido de NumPy limitado a un hilo. Python puro presenta un crecimiento mucho mayor respecto al tamaño del problema, mientras que las optimizaciones del compilador reducen considerablemente el tiempo de la implementación en C.

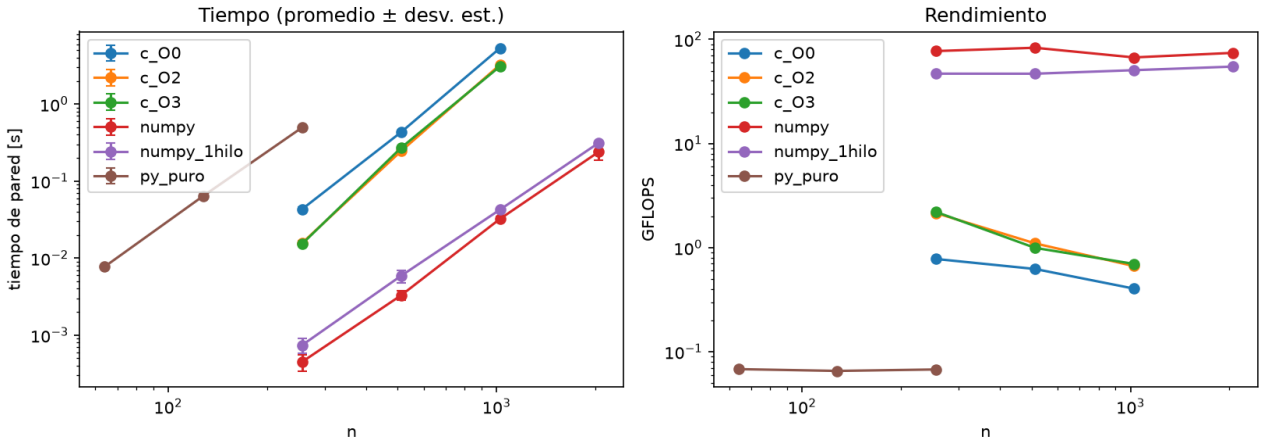



*Figura 1. Tiempo de ejecución promedio ± desviación estándar (izquierda) y rendimiento en GFLOPS (derecha) de las diferentes implementaciones.*

La gráfica de rendimiento (Figura 1, derecha) muestra una diferencia considerable entre las implementaciones. NumPy alcanza valores superiores a 80 GFLOPS, mientras que la mejor medición promedio de la implementación ingenua en C fue aproximadamente 2.20 GFLOPS. Python puro permanece alrededor de 0.067 GFLOPS.

**Nivel de optimización**

Para n = 1024, la versión sin optimización (-O0) tardó en promedio T(O0) = 5.313345 s y la versión compilada con -O3 -march=native T(O3) = 3.078917 s, por lo que el factor de aceleración fue T(O0) / T(O3) = 5.313345 / 3.078917 ≈ 1.73. Sin embargo, -O2 (3.231043 s) ya logra 1.64 veces: casi toda la mejora viene de optimizaciones escalares y no de la vectorización. Con -O0 el compilador lee y escribe en la pila cada variable (s, k y los índices) en cada iteración; con -O2 las mantiene en registros y simplifica el cálculo de índices (i*n + k se convierte en un puntero que avanza). En el ensamblador de -O3 se ve además que el compilador integró (inline) matmul y now dentro de main.

El ensamblador se generó con gcc -O3 -march=native -S matmul.c -o matmul_O3.s y se buscaron instrucciones vectoriales con grep (los listados completos están en el Anexo A). Al revisarlo se observa que no todas las instrucciones vectoriales encontradas pertenecen a la multiplicación:

- Los vmulpd con registros ymm (4 doubles) están en el ciclo que inicializa A y B: calcula i % 7 e i % 5 con multiplicaciones por constantes (vpmuldq), convierte a double (vcvtdq2pd) y multiplica por 0.5 y 0.25. Ese ciclo sí está vectorizado a 256 bits, pero queda fuera de la región medida.
- vfmadd132sd .LC17 aparece justo después de cada llamada a clock_gettime: es el cálculo ts.tv_sec + ts.tv_nsec * 1e-9 de la función now() (.LC17 = 1e-9).
- vfmadd231sd (FMA escalar) solo procesa el elemento sobrante cuando n es impar; con n = 256, 512 y 1024 no se ejecuta.
- El ciclo interno de k (etiquetas .L17 y .L27, una copia para el calentamiento y otra para las repeticiones) es el siguiente:

```
.L17:
  vmovsd    (%rax), %xmm0               # B[k][j]
  addq      $16, %rdx
  vmovhpd   (%rax,%rbx,8), %xmm0, %xmm0  # B[k+1][j]  (n·8 bytes más adelante)
  vmulpd    -16(%rdx), %xmm0, %xmm0     # A[i][k]·B[k][j] y A[i][k+1]·B[k+1][j]  (128 bits)
  addq      %rcx, %rax                  # avanza dos filas de B
  vaddsd    %xmm0, %xmm1, %xmm1         # s += primer producto   (escalar)
  vunpckhpd %xmm0, %xmm0, %xmm0
  vaddsd    %xmm1, %xmm0, %xmm1         # s += segundo producto  (escalar)
  cmpq      %rsi, %rdx
  jne       .L17
```

Es decir, la vectorización del producto es parcial: solo las multiplicaciones se hacen de dos en dos en registros de 128 bits; las sumas siguen siendo escalares y una detrás de otra, porque sin -ffast-math el compilador no puede reordenar sumas de punto flotante (cambiaría el redondeo). Además, como el ciclo recorre B por columnas, cada elemento se carga por separado y está a n·8 = 8 KiB del anterior en n = 1024; cada matriz ocupa 8 MiB y ya no cabe en los 6 MiB de L3. Por eso -O3 apenas mejora a -O2 (3.08 s contra 3.23 s) e incluso quedó ligeramente por detrás en n = 512 (0.270 s contra 0.245 s): el ciclo está limitado por los accesos a memoria y por la cadena de sumas sobre s, no por el número de instrucciones. Reordenar los ciclos a i-k-j, como hace matmul_pure.py, volvería contiguos los accesos a B y a C y permitiría una vectorización completa.

**Python puro contra NumPy**

Para n = 256 Python puro obtuvo $T_{Python}$ = 0.496554 s, mientras que NumPy obtuvo $T_{NumPy}$ = 0.0004502 s. La razón entre ambos tiempos es de 0.496554 / 0.0004502 ≈ 1103; por lo tanto, NumPy fue aproximadamente 1103 veces más rápido que Python puro.

Esta diferencia tiene principalmente dos causas. Primero, Python puro ejecuta los ciclos y operaciones aritméticas mediante el intérprete, provocando una sobrecarga considerable en cada iteración. NumPy, en cambio, delega la operación matricial a código compilado.

Segundo, NumPy utiliza bibliotecas BLAS optimizadas para operaciones matriciales. Estas implementaciones aprovechan técnicas como bloques para mejorar el aprovechamiento de caché, vectorización SIMD y ejecución mediante varios hilos.

**Paralelismo oculto de NumPy**

Para n = 2048, NumPy normal tardó $T_{NumPy}$ = 0.239884 s, mientras que NumPy limitado a un hilo obtuvo $T_{1\ hilo}$ = 0.313831 s; por lo tanto, 0.313831 / 0.239884 ≈ 1.31. La ejecución normal fue aproximadamente 1.31 veces más rápida.

En las cinco ejecuciones de NumPy para n = 2048, cpu_over_wall fue 4.04, 3.49, 3.96, 4.01 y 3.59; su promedio es aproximadamente de 3.82. Esto indica que NumPy utilizó aproximadamente 4 hilos efectivos sin haberlos solicitado explícitamente. En cambio, numpy_1hilo obtuvo valores de cpu_over_wall cercanos a 1.00, lo que confirma la ejecución con un solo hilo.

Por lo tanto, utilizar NumPy normalmente no garantiza una ejecución secuencial, ya que BLAS puede utilizar paralelismo internamente. Para establecer una línea base secuencial válida es necesario limitar explícitamente los hilos mediante “OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1, MKL_NUM_THREADS=1”.

**Pico teórico**

Para calcularlo debemos emplear: Pico ≈ núcleos × frecuencia × FLOP/ciclo. Siguiendo la tabla de la práctica se utilizan aproximadamente 32 FLOP/ciclo por núcleo (fila «AVX-512 + FMA, una unidad», porque el i3-1115G4 soporta AVX-512F y FMA). Con 2 núcleos y 3.00 GHz tenemos 2 × 3.00 × 32 = 192 GFLOPS. La mejor medición promedio en C fue de 2.2002 GFLOPS, por lo que nos da un aproximado de 1.15 %. La mejor medición promedio de NumPy con todos los hilos fue 83.0492 GFLOPS, con un 43.25 %. Así, bajo esta suposición, C alcanzó aproximadamente 1.15 % del pico teórico y NumPy aproximadamente 43.3 %.

La implementación de C permanece lejos del pico incluso con -O3 porque el algoritmo no aprovecha eficientemente la jerarquía de memoria. En particular, los accesos a la matriz B producen un patrón poco favorable para la caché. Además, -O3 puede optimizar y vectorizar operaciones, pero no transforma automáticamente el algoritmo ingenuo en una implementación especializada comparable con BLAS, que emplea bloqueo de matrices, vectorización y otras optimizaciones.

**Variabilidad**

La mayor variabilidad se presentó en la ejecución de NumPy con n = 256, donde se obtuvo un tiempo promedio de 0.0004502 s y una desviación estándar de 0.00010719 s. Al calcular la desviación estándar relativa, se obtiene aproximadamente un 23.81 %, valor que supera el 10 % recomendado en la práctica como límite para considerar estable una medición. Esto puede explicarse porque el tiempo de ejecución en este caso es extremadamente pequeño, alrededor de 0.45 ms, por lo que cualquier interrupción mínima del sistema operativo, proceso en segundo plano, cambio momentáneo en la frecuencia del procesador o gestión interna de los hilos de BLAS puede representar una variación porcentual considerable. También se observó una variabilidad alta en NumPy con n = 2048, con aproximadamente 22.52 % de desviación relativa, debido principalmente a que una de las cinco ejecuciones tardó 0.330360 s, mientras que las demás se mantuvieron aproximadamente entre 0.189 y 0.236 s. Esto sugiere que durante esa repetición pudo existir una interferencia temporal del sistema. Por esta razón, los resultados deben interpretarse considerando que el rendimiento puede verse afectado por procesos externos y por la planificación de recursos del sistema.

**E) Acceso a GPU (Google Colab y Kaggle)**

Para evaluar el rendimiento de aceleración por hardware, se ejecutaron pruebas de multiplicación de matrices utilizando la biblioteca CuPy en un entorno de Google Colab equipado con una tarjeta gráfica NVIDIA Tesla T4

(16 GB de VRAM). La verificación del entorno ejecutando el comando nvidia-smi confirmó la siguiente configuración:

```
Wed Oct  7 02:15:41 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  Tesla T4                       Off |   00000000:00:04.0 Off |                    0 |
| N/A   50C    P8             11W /   70W |       0MiB /  15360MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
Build cuda_13.0.r13.0/compiler.36424714_0
Tesla T4 | CuPy 14.0.1
```

*Figura 2. Salida de nvidia-smi en Google Colab (7 de octubre de 2026).*

**Tabla comparativa de rendimiento (CPU vs. GPU)**

Se midieron los tiempos de ejecución de la multiplicación de matrices C = A · B en precisión simple (float32), que es el formato en el que la T4 alcanza su mayor rendimiento, bajo tres modalidades: CPU de Colab, GPU con sincronización estricta (sync), y el tiempo medido de forma incorrecta sin sincronizar (sin sync). Los resultados se muestran en la Tabla 4.

*Tabla 4. Tiempos de la multiplicación de matrices en la CPU (NumPy) y en la GPU (CuPy) de Google Colab, con y sin sincronización.*

| Tamaño del problema (n) | Tiempo CPU (s) | Tiempo GPU (s) | Speedup (CPU/GPU) | Tiempo «sin sync» (s) |
|---|---|---|---|---|
| 256 | 0.0064 | 0.00013 | 49.7x | 0.000055 |
| 1024 | 0.0362 | 0.00142 | 25.5x | 0.000067 |
| 4096 | 1.249 | 0.04277 | 29.2x | 0.000035 |

**CPU contra GPU**

Para n = 256 la T4 fue ≈ 49.7 veces más rápida que NumPy en la CPU de Colab, y para n = 4096, ≈ 29.2 veces. En nuestras mediciones la GPU ganó incluso con n pequeño, al contrario de lo que plantea la pregunta, pero los mismos datos muestran por qué ese resultado no se generaliza:

1. **La GPU está muy subutilizada con n pequeño.** En n = 256 el producto son $2 \cdot 256^3 \approx 33.5$ millones de operaciones y tardó 0.13 ms, es decir, ≈ 0.26 TFLOPS: apenas ~3 % del pico de la T4 en float32 (8.1 TFLOPS). En n = 4096 alcanzó ≈ 3.2 TFLOPS (~40 % del pico). Con matrices chicas no hay trabajo suficiente para ocupar sus 2560 núcleos CUDA y pesan los costos fijos de lanzar el kernel y sincronizar.
2. **La medición no incluye las copias entre CPU y GPU.** cp.asarray se ejecuta fuera de la región medida y C se queda en la GPU. En un uso real hay que copiar A y B (256 KiB cada una) y traer C de regreso por PCIe, y eso cuesta del mismo orden que los 0.13 ms del cálculo.
3. **La CPU de Colab rindió muy poco en n = 256.** Obtuvo ≈ 5.2 GFLOPS, contra ≈ 59 GFLOPS en n = 1024 y ≈ 110 GFLOPS en n = 4096 (Colab gratuito suele asignar solo 2 núcleos virtuales). Por eso el speedup en n = 256 es mayor que en n = 1024 (25.5×). Como referencia, NumPy en la laptop de Rubén resolvió n = 256 en 0.45 ms aun en doble precisión: frente a eso la T4 sería apenas ≈ 3.5 veces más rápida, y al sumar las copias la ventaja prácticamente desaparece.

Sin synchronize() la GPU “tardó” entre 35 y 67 µs en los tres tamaños, aunque de n = 256 a n = 4096 el trabajo crece 4096 veces. Ese número es falso porque CuPy lanza los kernels de forma asíncrona: Ag @ Bg regresa en cuanto el kernel queda en la cola de la GPU, y el reloj se lee antes de que termine, así que solo se mide el costo de lanzarlo. En n = 4096 implicaría ≈ 3,900 TFLOPS, unas 480 veces el pico de la T4, lo cual es físicamente imposible. Al llamar a cp.cuda.Device().synchronize() antes de leer el reloj, la CPU espera a que la GPU termine y se obtienen los 42.8 ms reales.

## Problemas encontrados

La Tabla 5 resume los problemas que aparecieron durante la práctica, su causa y su solución.

*Tabla 5. Problemas encontrados, su causa y cómo se resolvieron.*

| Problema | Causa | Cómo se resolvió |
|---|---|---|
| Variabilidad elevada en la primera corrida de run_all.sh. | Aplicaciones y procesos en segundo plano compitiendo por la CPU en una laptop de solo 2 núcleos físicos. | Se cerraron las aplicaciones innecesarias, se dejó el equipo conectado a la corriente y se repitió la corrida completa. Las tablas del reporte corresponden a esa segunda ejecución. |
| Aun en la segunda corrida, NumPy superó el 10 % de desviación relativa en n = 256 (23.8 %) y n = 2048 (22.5 %). | En n = 256 cada producto dura ≈ 0.45 ms, así que cualquier interrupción del sistema pesa mucho en porcentaje; además, la primera repetición dio cpu_over_wall = 16.45, un valor imposible con 4 hilos lógicos. En n = 2048 una sola repetición tardó 0.330 s contra 0.189–0.236 s de las demás. | Se investigó la causa, como pide el protocolo, y se documentó en la sección de variabilidad. Para el proyecto conviene usar más repeticiones (o hyperfine) y reportar también el mínimo en las corridas muy cortas. |
| NumPy reportó hasta 83 GFLOPS en la configuración por defecto sin haber pedido paralelismo. | OpenBLAS usa varios hilos de forma automática: cpu_over_wall promedió 3.82 en n = 2048, es decir, ≈ 4 hilos. | Se midió también numpy_1hilo con OMP_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1 MKL_NUM_THREADS=1 (cpu_over_wall ≈ 1.00). Esa es la configuración que debe usarse para la línea base V1 del proyecto. |
| En WSL2 no se pudo leer el gobernador ni la frecuencia de la CPU; la frecuencia turbo y el kernel quedaron como «por verificar» en las fichas. | WSL2 corre sobre el hipervisor de Windows (lscpu reporta «Hypervisor vendor: Microsoft») y no expone cpufreq: el comando de scaling_governor no imprimió nada. | Se tomó la frecuencia de la especificación del fabricante (i3-1115G4: 3.0 / 4.1 GHz; i5-12400F: 2.5 / 4.4 GHz) y el kernel se consulta con uname -r. Para el pico teórico se usó la frecuencia base de 3.00 GHz que reporta lscpu. |
| No hay GPU NVIDIA disponible dentro de WSL2 (nvidia-smi: command not found). | Las máquinas de Rubén y Carlos no tienen una GPU NVIDIA accesible desde WSL2. | La Parte E se ejecutó en Google Colab con una Tesla T4 (16 GB, CUDA 13.0, CuPy 14.0.1). Kaggle queda como respaldo para cuando Colab no asigne GPU. |
| Sin synchronize() la GPU parecía tardar solo 35–67 µs, incluso con n = 4096. | CuPy lanza los kernels de forma asíncrona: sin sincronizar, el reloj se detiene cuando el kernel apenas se encoló, no cuando terminó. | Se midió con cp.cuda.Device().synchronize() antes de leer el reloj (42.8 ms en n = 4096). Los tiempos «sin sync» se reportan solo como ejemplo de medición incorrecta. |

## Conclusiones

La principal lección de la práctica es que antes de paralelizar hay que saber medir. Con una corrida de calentamiento, cinco repeticiones y un reloj monotónico obtuvimos una línea base sólida, aunque dos pruebas de NumPy superaron el 10 % de desviación. Sin usar hilos, pasar de -O0 a -O3 en la versión ingenua (n = 1024) dio una aceleración de 1.73 veces, pero el rendimiento se quedó en 2.2 GFLOPS (≈ 1.15 % del pico teórico), porque el acceso por columnas a B aprovecha mal la caché. Con NumPy la diferencia fue enorme: en n = 256 fue ≈ 1103 veces más rápido que Python puro gracias a BLAS, aunque usó 4 hilos sin pedirlo, así que la V1 del proyecto debe medirse con un solo hilo. En GPU, la T4 fue de 25 a 50 veces más rápida que la CPU de Colab, pero sin sincronizar los tiempos medidos no son reales.

## Uso de IA

Durante el desarrollo de la práctica se utilizaron herramientas de IA generativa, como ChatGPT y Claude (Anthropic), como apoyo para comprender las instrucciones de ejecución para WSL2, interpretar el funcionamiento de los scripts proporcionados, explicar conceptos relacionados con la medición de tiempos, optimización del compilador, vectorización, paralelismo interno de NumPy y análisis de rendimiento.

Los resultados experimentales no fueron generados por la IA. Las mediciones fueron obtenidas directamente mediante los programas y scripts proporcionados en la práctica, principalmente run_all.sh y plot.py.

De manera manual se verificó la correcta ejecución de los programas, la generación de los archivos c.csv, py_puro.csv, numpy.csv y resumen.csv, así como las gráficas generadas. También se revisaron los valores utilizados en los cálculos directamente en los archivos de resultados.

Claude también se utilizó como apoyo para redactar el borrador de la tabla de problemas encontrados, las conclusiones y las respuestas a las preguntas guía 1 y 6; los valores citados se verificaron contra los archivos CSV, el ensamblador generado (matmul_O3.s) y la salida del notebook de GPU.

## Tabla de contribuciones

*Tabla 6. Contribución individual de cada integrante.*

| Alumno | Contribución | Porcentaje |
|---|---|---|
| Salinas Padilla Rubén Iván | Sección C y D | 40% |
| Arce Padilla Carlos | Sección E | 30% |
| Montes Olguin José Alberto | Sección F | 30% |

## Anexo A. Listados del ensamblador (matmul_O3.s)

Salida de los comandos grep ejecutados sobre matmul_O3.s, generado con gcc -O3 -march=native -S matmul.c -o matmul_O3.s en la laptop de Salinas Padilla Rubén Iván (gcc 15.2.0).

```
$ grep -Ei 'vfmadd|vmulpd|vaddpd' matmul_O3.s
        vmulpd  %ymm9, %ymm3, %ymm3
        vmulpd  %ymm9, %ymm1, %ymm1
        vmulpd  %ymm8, %ymm1, %ymm1
        vmulpd  %ymm8, %ymm0, %ymm0
        vmulpd  %xmm3, %xmm2, %xmm2
        vmulpd  %xmm3, %xmm0, %xmm0
        vmulpd  %xmm2, %xmm0, %xmm0
        vmulpd  %xmm2, %xmm0, %xmm0
        vmulpd  -16(%rdx), %xmm0, %xmm0
        vfmadd231sd     0(%r13,%rax,8), %xmm7, %xmm1
        vfmadd132sd     .LC17(%rip), %xmm0, %xmm1
        vmulpd  -16(%rcx), %xmm2, %xmm2
        vfmadd231sd     0(%r13,%rdx,8), %xmm6, %xmm0
        vfmadd132sd     .LC17(%rip), %xmm1, %xmm0

$ grep -Ei 'ymm|zmm|xmm|vfm|vmul|vadd' matmul_O3.s | head -30
        vmovdqa .LC1(%rip), %ymm2
        vmovdqa .LC4(%rip), %ymm10
        vpbroadcastd    %ecx, %ymm7
        vbroadcastsd    .LC7(%rip), %ymm9
        vpbroadcastd    %ecx, %ymm4
        vbroadcastsd    .LC9(%rip), %ymm8
        vpbroadcastd    %ecx, %ymm6
        vpsrlq  $32, %ymm2, %ymm5
        vpmuldq %ymm7, %ymm2, %ymm1
        vpmuldq %ymm7, %ymm5, %ymm0
        vpmuldq %ymm4, %ymm2, %ymm3
        vpmuldq %ymm4, %ymm5, %ymm5
        vpermt2d        %ymm0, %ymm10, %ymm1
        vpermt2d        %ymm5, %ymm10, %ymm3
        vpsrad  $1, %ymm1, %ymm1
        vpaddd  %ymm2, %ymm3, %ymm3
        vpslld  $2, %ymm1, %ymm0
        vpsrad  $2, %ymm3, %ymm3
        vpaddd  %ymm1, %ymm0, %ymm0
        vpslld  $3, %ymm3, %ymm1
        vpsubd  %ymm0, %ymm2, %ymm0
        vpsubd  %ymm3, %ymm1, %ymm1
        vpsubd  %ymm1, %ymm2, %ymm1
        vpaddd  %ymm6, %ymm2, %ymm2
        vcvtdq2pd       %xmm1, %ymm3
        vmulpd  %ymm9, %ymm3, %ymm3
        vextracti32x4   $0x1, %ymm1, %xmm1
        vcvtdq2pd       %xmm1, %ymm1
        vmulpd  %ymm9, %ymm1, %ymm1
        vmovupd %ymm3, 0(%r13,%rax)
```

## Anexo B. Salida de la verificación del entorno (Parte A)

Salida de terminal de cada integrante, tal como se obtuvo al ejecutar los comandos de verificación de la Parte A. La salida completa, junto con la de la Parte B, también está en practicas/practica1/entorno.txt del repositorio.

### Salinas Padilla Rubén Iván

```
(cp-env) ruben@LAPTOP-EHOSFET3:~$ gcc --version | head -1
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
(cp-env) ruben@LAPTOP-EHOSFET3:~$ g++ --version | head -1
g++ (Ubuntu 15.2.0-16ubuntu1) 15.2.0
(cp-env) ruben@LAPTOP-EHOSFET3:~$ python3 --version && python3 -c "import numpy; print('numpy', numpy.__version__)"
Python 3.14.4
numpy 2.5.3
(cp-env) ruben@LAPTOP-EHOSFET3:~$ git --version
git version 2.53.0
(cp-env) ruben@LAPTOP-EHOSFET3:~$ echo | gcc -fopenmp -dM -E - | grep -i openmp
#define _OPENMP 201511
```

```
(cp-env) ruben@LAPTOP-EHOSFET3:~$ nano hello_omp.c
(cp-env) ruben@LAPTOP-EHOSFET3:~$ gcc -fopenmp hello_omp.c -o hello_omp
(cp-env) ruben@LAPTOP-EHOSFET3:~$ ./hello_omp
Hola desde el hilo 1 de 4
Hola desde el hilo 3 de 4
Hola desde el hilo 2 de 4
Hola desde el hilo 0 de 4
```

### Arce Padilla Carlos

```
(cp-env) carlos@DESKTOP-FGPKF6Q:~$ gcc --version | head -1
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
(cp-env) carlos@DESKTOP-FGPKF6Q:~$ g++ --version | head -1
g++ (Ubuntu 15.2.0-16ubuntu1) 15.2.0
(cp-env) carlos@DESKTOP-FGPKF6Q:~$ python3 --version && python3 -c "import numpy; print('numpy', numpy.__version__)"
Python 3.14.4
numpy 2.5.3
(cp-env) carlos@DESKTOP-FGPKF6Q:~$ git --version
git version 2.53.0
(cp-env) carlos@DESKTOP-FGPKF6Q:~$ gcc -fopenmp -dM -E - < /dev/null | grep -i openmp
#define _OPENMP 201511
(cp-env) carlos@DESKTOP-FGPKF6Q:~$ cat hello_omp.c
#include <stdio.h>
#include <omp.h>
int main(void) {
    #pragma omp parallel
    {
        printf("Hola desde el hilo %d de %d\n", omp_get_thread_num(), omp_get_num_threads());
    }
    return 0;
}
(cp-env) carlos@DESKTOP-FGPKF6Q:~$ gcc -fopenmp hello_omp.c -o hello_omp
(cp-env) carlos@DESKTOP-FGPKF6Q:~$ OMP_NUM_THREADS=4 ./hello_omp
Hola desde el hilo 3 de 4
Hola desde el hilo 1 de 4
Hola desde el hilo 2 de 4
Hola desde el hilo 0 de 4
```

### Montes Olguin José Alberto

```
(cp-env) alberto@Quetzal:~$ gcc --version | head -1
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
(cp-env) alberto@Quetzal:~$ g++ --version | head -1
g++ (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
(cp-env) alberto@Quetzal:~$ python3 --version && python3 -c "import numpy; print('numpy', numpy.__version__)"
Python 3.12.3
numpy 2.5.3
(cp-env) alberto@Quetzal:~$ git --version
git version 2.43.0
(cp-env) alberto@Quetzal:~$ echo | gcc -fopenmp -dM -E - | grep -i openmp
#define _OPENMP 201511
(cp-env) alberto@Quetzal:~/p1$ gcc -fopenmp hello_omp.c -o hello_omp
(cp-env) alberto@Quetzal:~/p1$ OMP_NUM_THREADS=1 ./hello_omp
Hola desde el hilo 0 de 1
(cp-env) alberto@Quetzal:~/p1$ OMP_NUM_THREADS=4 ./hello_omp
Hola desde el hilo 0 de 4
Hola desde el hilo 2 de 4
Hola desde el hilo 1 de 4
Hola desde el hilo 3 de 4
```